

Fibonacci Numbers

The Fibonacci numbers can be defined as follows:

```
F_0 = 0
F_1 = 1
F_n = F_{n-2} + F_{n-1}
```

Your task is to calculate the value of F_n for a given n .

Input

The only input line has an integer n .

Output

Print the value of F_n modulo 10^9+7 .

Constraints

$0 \leq n \leq 10^{18}$

Example

Input:

10

Output:

55

Why normal DP is impossible

If $n \leq 10^6$, you'd do:

```
for i = 2 to n:
    F[i] = F[i-1] + F[i-2]
```

But here:

- n can be 10^{18}
- You cannot loop that many times

So we need:

- Logarithmic time
- That immediately hints: binary exponentiation
- But what are we exponentiating? → a transition

Think in terms of "state", not "formula"

This is the most important mental shift.

Ask:

What minimum information do I need to compute the next Fibonacci number?

Answer:

- $F(n)$
- $F(n-1)$

So define a state vector:

$$S(n) = \begin{bmatrix} F(n) \\ F(n-1) \end{bmatrix}$$

Why matrix exponentiation works

We now have:

$$S(n+1) = M \cdot S(n)$$

Apply repeatedly:

- $S(2) = M \cdot S(1)$
- $S(3) = M^2 \cdot S(1)$
- ...
- $S(n) = M^{(n-1)} \cdot S(1)$

This is the entire idea.

Define the base state correctly

We know:

```
F_1 = 1
F_0 = 0
```

So:

$$S(1) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Thus:

$$F_n = \text{first element of } M^{n-1} \cdot S(1)$$

Special cases:

- If $n = 0 \rightarrow$ answer is 0
- If $n = 1 \rightarrow$ answer is 1

Code

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const ll MOD = 1000000007;

// 2x2 matrix
struct Matrix {
    ll a[2][2];
};

// Multiply two 2x2 matrices
Matrix multiply(Matrix A, Matrix B) {
    Matrix C;
    memset(C.a, 0, sizeof(C.a));

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            for (int k = 0; k < 2; k++) {
                C.a[i][j] = (C.a[i][j] + A.a[i][k] * B.a[k][j]) % MOD;
            }
        }
    }
    return C;
}

// Fast exponentiation of matrix
Matrix matrix_power(Matrix base, long long exp) {
    // Identity matrix
    Matrix result;
    result.a[0][0] = 1; result.a[0][1] = 0;
    result.a[1][0] = 0; result.a[1][1] = 1;

    while (exp > 0) {
        if (exp & 1)
            result = multiply(result, base);
        base = multiply(base, base);
        exp >>= 1;
    }
    return result;
}

int main() {
    long long n;
    cin >> n;

    if (n == 0) {
        cout << 0 << "\n";
        return 0;
    }
    if (n == 1) {
        cout << 1 << "\n";
        return 0;
    }

    Matrix M;
    M.a[0][0] = 1; M.a[0][1] = 1;
    M.a[1][0] = 1; M.a[1][1] = 0;

    Matrix Mn = matrix_power(M, n - 1);

    // S(1) = [1, 0]
    ll Fn = (Mn.a[0][0] * 1 + Mn.a[0][1] * 0) % MOD;
    cout << Fn << "\n";

    return 0;
}
```